

32-bit ARM ALU Using Verilog HDL

Designed by

Md. Luman Hossain

Course

VLSI Design Lab

Development Environment

- Verilog HDL
- Cadence NC Launch
- Cadence NCSim

Project Overview

Objective

This project focuses on the design, implementation, and functional verification of a 32-bit ARM Arithmetic Logic Unit (ALU) using Verilog HDL. The design was simulated and verified in Cadence NCSim to gain practical experience in RTL design and digital hardware verification.

RTL Implementation :

```

module ALU32Bit(
    input  [31:0] A, B,
    input  [1:0] opcode,
    output reg [31:0] R,
    output reg Z, N, C, V
);
    reg [32:0] temp;
    always @(*) begin
        R = 32'b0;
        C = 1'b0;
        V = 1'b0;
        if (opcode == 2'b00) begin
            temp = {1'b0, A} + {1'b0, B};
            R = temp[31:0];
            C = temp[32]; // Carry out
            V = (A[31] & B[31] & ~R[31]) | (~A[31] & ~B[31] & R[31]);
        end

        if (opcode == 2'b01) begin
            temp = {1'b0, A} - {1'b0, B};
            R = temp[31:0];
            C = ~temp[32]; // Borrow = NOT(carry)
            V = (A[31] & ~B[31] & ~R[31]) | (~A[31] & B[31] & R[31]);
        end

        if (opcode == 2'b10) begin
            R = A & B;
            C = 1'b0;
            V = 1'b0;
        end
        if (opcode == 2'b11) begin
            R = A | B;
            C = 1'b0;
            V = 1'b0;
        end
        Z = (R == 32'b0);
        N = R[31];
    end
endmodule

```


Functional Verification : A Verilog testbench was developed to verify the correctness of all ALU operations under different input combinations.

```

module ALU32Bit_tb();
    `timescale 1ns/1ps

    reg [31:0] A, B;
    reg [1:0] opcode;
    wire [31:0] R;
    wire Z, N, C, V;
    reg clk;
    reg [31:0] R_sample;
    ALU32Bit uut (.A(A), .B(B), .opcode(opcode), .R(R), .Z(Z), .N(N), .C(C), .V(V));

    initial
    begin
        clk = 0;

    end
    always #5 clk = ~clk;

    initial begin
        A = 0; B = 0; opcode = 2'b00; #12;

        @(posedge clk);
        A = 10; B = 5; opcode = 2'b00;
        @(posedge clk); #1;

        @(posedge clk);
        A = 32'hF0F0F0F0; B = 32'h0F0F0F0F; opcode = 2'b10;
        @(posedge clk); #1;

        @(posedge clk);
        A = 32'hAAAA5555; B = 32'h0F0F0F0F; opcode = 2'b11;
        @(posedge clk); #1;

        #40;
        $finish;
    end
    always @(posedge clk) begin
        R_sample <= R;
    end
endmodule

```

Simulation Results : The simulation confirms that the ALU correctly performs the intended arithmetic and logical operations. The outputs change according to the selected control signal, validating the RTL implementation.

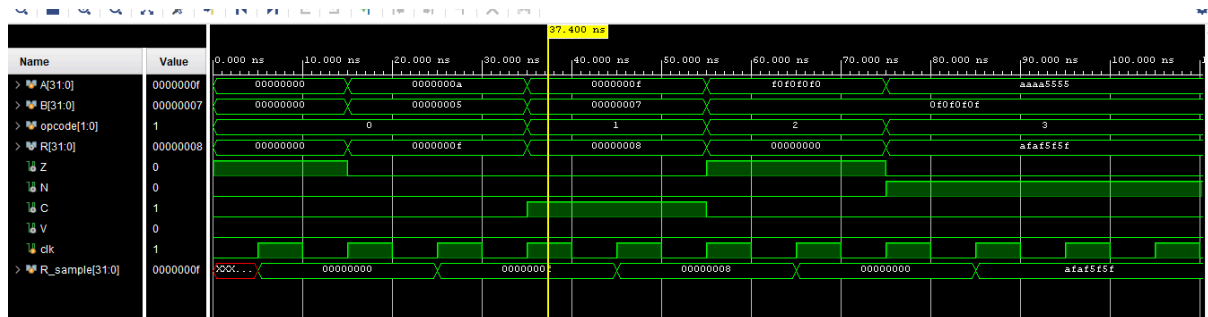


Figure 2. Simulation Waveform

Conclusion

The successful implementation and verification of the **32-bit ARM ALU** demonstrate the effectiveness of RTL design and simulation in developing reliable digital hardware. By implementing arithmetic and logical operations, creating a comprehensive testbench, and validating the design through simulation waveforms, this project provided valuable hands-on experience in **Verilog HDL**, **RTL modeling**, **functional verification**, and **digital system design**. These skills form a strong foundation for advanced VLSI and ASIC design projects.

Skills Gained

Technical Skills

RTL Design

- ✓ Verilog HDL
- ✓ Combinational Logic Design
- ✓ 32-bit ARM ALU Implementation

Verification

- ✓ Testbench Development
- ✓ Functional Verification
- ✓ Waveform Analysis

EDA Tools

- ✓ Cadence NC Launch
- ✓ Cadence NCSim

Key Learning Outcomes

- Designed a synthesizable **32-bit ARM ALU** using Verilog HDL.
- Verified ALU functionality through simulation and waveform analysis.
- Gained practical experience in RTL modeling and digital hardware verification.
- Strengthened understanding of the RTL design flow used in VLSI development